

Enterprise EKS microservices setup — verified & formatted

This is the full infrastructure build: VPC → data layer (RDS/Redis/Kafka) → EKS cluster → in-cluster networking (ALB Ingress) → secrets management → application deployment. Corrections from the original notes are marked **[FIXED]** inline. The "lessons learned" table at the end is kept as-is — it's a genuinely useful record and every entry checks out.

Phase 1 — VPC and networking foundation

1.1 Create the VPC

- VPC Dashboard → Your VPCs → Create VPC → "VPC only"
- Name: `microservices-vpc`
- IPv4 CIDR: `10.0.0.0/16` (65,536 IPs)
- Enable **DNS resolution** and **DNS hostnames**

1.2 Subnets (multi-AZ)

Name	AZ	Type	CIDR	Purpose
public-subnet-1a	1a	Public	10.0.1.0/24	ALB, NAT Gateway, Bastion
public-subnet-1b	1b	Public	10.0.2.0/24	ALB, NAT Gateway
private-app-subnet-1a	1a	Private	10.0.3.0/24	EKS worker nodes / pods
private-app-subnet-1b	1b	Private	10.0.4.0/24	EKS worker nodes / pods
private-data-subnet-1a	1a	Private	10.0.5.0/24	RDS, Redis, Kafka
private-data-subnet-1b	1b	Private	10.0.6.0/24	RDS standby, Redis nodes

Required EKS tags (skip these and the ALB controller silently fails to find subnets later):

- Public subnets: `kubernetes.io/role/elb = 1`
- Private app subnets: `kubernetes.io/role/internal-elb = 1`

Note: these are planning-time CIDRs. Before writing any Security Group rule later, re-check your subnets' *actual* CIDR ranges in the console — plans drift from reality (see the lessons table, #30-32, where planned `/24`s turned into real `/20`s and broke an RDS security group rule).

1.3 Gateways & routing

- **Internet Gateway:** create, attach to `microservices-vpc`
- **NAT Gateway:** one in `public-subnet-1a` with an Elastic IP (a second in 1b is the HA option — skip it for learning, add it later)
- **Public route table:** associate with both public subnets → `0.0.0.0/0` → IGW ("anything I don't recognize internally, send out to the internet")
- **Private route table:** associate with all private subnets → `0.0.0.0/0` → NAT Gateway (*outbound-only — nothing can initiate a connection back in*)

1.4 Security groups

Security groups are **stateful** — a rule you allow inbound automatically allows the matching response traffic back out, so outbound can usually stay at the default "allow all."

Security Group	Inbound rule
<code>Database-SG</code>	Port 5432/3306, source = private app subnet CIDRs only
<code>Redis-SG</code>	Port 6379, source = private app subnets only
<code>Kafka-SG</code>	Ports 9092/9094, source = private app subnets only
<code>EKS-Nodes-SG</code>	Created during cluster setup — needs outbound to the data layer

[FIXED — production tip, matches your own lesson #32]: once your app is running, prefer referencing the **EKS Node Security Group directly** as the source (SG-to-SG rule) instead of a CIDR block. CIDR-based rules break silently the moment your actual subnet range doesn't match what you assumed; SG references don't have that failure mode.

Phase 2 — Data layer

2.1 RDS (PostgreSQL/MySQL)

- Subnet group: both private-data subnets
- Multi-AZ deployment (standby in AZ2)
- VPC: `microservices-vpc`, attach `Database-SG`
- **Public access:** No

2.2 ElastiCache (Redis) & MSK (Kafka)

- Redis: place in data subnets, attach `Redis-SG`, note the primary endpoint

- MSK: 2 brokers (one per AZ), attach `Kafka-SG` — provisioning takes ~45 minutes, plan around that

2.3 Secrets Manager

- Create one secret (key/value): `DB_USERNAME`, `DB_PASSWORD`, `REDIS_AUTH`
 - Note the ARN — needed for the IAM policy in Phase 4
-

Phase 3 — EKS cluster & compute

3.1 IAM roles

- **Cluster role:** `AmazonEKSClusterPolicy`
- **Node group role:** `AmazonEKSWorkerNodePolicy` + `AmazonEC2ContainerRegistryReadOnly` + `AmazonEKS_CNI_Policy`

3.2 Create the cluster

- Name: `demo-cluster`, attach the cluster role
- VPC: `microservices-vpc`
- Endpoint access: **Public and Private**
- ~15 minutes to become Active

Networking subnet choice: your notes select *only* the private app subnets for the cluster's own VPC config. This can work, but the safer default most teams use is to include **both public and private subnets** in the cluster's VPC config (not just the node group) — it avoids edge cases later when the ALB controller or other AWS integrations expect public subnets to be part of the cluster's known subnet list.

Add-ons to select during creation

Add-on	Category	What it does
Amazon VPC CNI	Networking	Gives pods real AWS VPC private IPs
CoreDNS	Service discovery	Internal DNS — lets pods resolve each other by name
kube-proxy	Networking	Maintains Service load-balancing rules on nodes
Amazon EBS CSI Driver	Storage	Dynamically attaches EBS volumes to pods
EKS Pod Identity Agent	Security	Maps IAM roles to service accounts — [NOTE] simpler alternative to IRSA below; you installed this but didn't

Add-on	Category	What it does
		end up using it — worth trying for your <i>next</i> role instead of the OIDC/trust-policy route
Amazon EFS CSI Driver	Storage	Shared multi-pod file storage via EFS
Amazon CloudWatch Observability	Monitoring	Auto-collects metrics/logs/traces to CloudWatch
AWS GuardDuty Agent	Security	Runtime threat detection inside the cluster

3.3 Managed node group

- Amazon Linux 2, `t3.medium`/`t3.large`
- Desired 2, min 2, max 4
- Place in private app subnets

3.4 Connect kubectl

```
bash
aws eks update-kubeconfig --region <your-region> --name demo-cluster
```

Phase 4 — In-cluster networking & add-ons

4.1 AWS Load Balancer Controller

What it does, plainly: it watches for `kind: Ingress` objects, and when it sees one, calls the AWS API to create the real ALB, target groups, listener rules, and security groups for you. This is the bridge between "AWS knows about ALBs" and "Kubernetes knows about pods" — nothing else provides that connection automatically.

Step 1 — IAM policy for the controller:

```
bash
curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-c
aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam_policy.json
```

Step 2 — OIDC provider (check first — you only need one per cluster):

IAM → Identity providers → look for a URL ending in `oidc.eks.`

`<region>.amazonaws.com/id/<your-id>`. If missing, add it: provider type OpenID

Connect, that same URL, audience `sts.amazonaws.com`.

Step 3 — IAM role for the controller, with this trust policy (replace account ID and OIDC id):

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::<ACCOUNT_ID>:oidc-provider/oidc.eks.us-east-1.amazonaws.com/id/<OIDC_ID>"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "oidc.eks.us-east-1.amazonaws.com/id/<OIDC_ID>:aud": "sts.amazonaws.com",
          "oidc.eks.us-east-1.amazonaws.com/id/<OIDC_ID>:sub": "system:serviceaccount:kube-system:aws-load-balancer-controller"
        }
      }
    }
  ]
}
```

Name it `AmazonEKSLoadBalancerControllerRole`.

Step 4 — service account + annotation (manual, non-eksctl route):

```
bash

kubectl create serviceaccount aws-load-balancer-controller -n kube-system

kubectl annotate serviceaccount aws-load-balancer-controller \
  -n kube-system \
  eks.amazonaws.com/role-arn=<AmazonEKSLoadBalancerControllerRole ARN>
```

This annotation is IRSA itself: *"any pod using this service account may assume this IAM role."*

Step 5 — install via Helm:

```
bash
```

```
helm repo add eks https://aws.github.io/eks-charts
```

```
helm repo update
```

```
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=k8-prod \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set region=us-east-1 \
  --set vpcId=<your-vpc-id>
```

Step 6 — subnet auto-discovery tags (the #1 cause of "no eligible subnets" errors):

Every subnet needs **two** tags:

- `kubernetes.io/cluster/<cluster-name> = shared` (or `owned`)
- Public subnets: `kubernetes.io/role/elb = 1` — Private subnets: `kubernetes.io/role/internal-elb = 1`

Without both, `type: LoadBalancer` / Ingress creation fails with an unhelpful "cannot find eligible subnets" error.

4.2 Secrets: AWS Secrets Manager → EKS

The flow: `AWS Secrets Manager → External Secrets Operator → Kubernetes Secret → Pod environment variables`

Install the operator:

```
bash
```

```
helm repo add external-secrets https://charts.external-secrets.io
```

```
helm repo update
```

```
helm install external-secrets external-secrets/external-secrets \
  -n external-secrets --create-namespace
```

[FIXED — this replaces the original "attach to Node Role" step, which was too broad]: give the operator its *own* scoped IAM identity via IRSA, not the shared node role.

```
bash
```

```
# 1. IAM policy – scoped to reading secrets only
cat > external-secrets-policy.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["secretsmanager:GetSecretValue", "secretsmanager:DescribeSecret"],
      "Resource": "*"
    }
  ]
}
EOF

aws iam create-policy \
  --policy-name ExternalSecretsPolicy \
  --policy-document file:///external-secrets-policy.json

# 2. IAM role bound to the external-secrets service account (via eksctl, one
eksctl create iamserviceaccount \
  --cluster k8-prod \
  --namespace external-secrets \
  --name external-secrets \
  --attach-policy-arn <ExternalSecretsPolicy ARN> \
  --approve \
  --region us-east-1

kubectl rollout restart deployment external-secrets -n external-secrets
```

SecretStore — this defines *how to connect* to AWS Secrets Manager:

```
yaml

apiVersion: external-secrets.io/v1 # [FIXED] v1beta1 is deprecated since ES
kind: SecretStore
metadata:
  name: aws-secretsmanager
  namespace: production
spec:
  provider:
    aws:
      service: SecretsManager
      region: us-east-1
```

ExternalSecret — this defines *which keys to actually pull*:

```
yaml

apiVersion: external-secrets.io/v1 # [FIXED]
kind: ExternalSecret
metadata:
  name: rds-secret
  namespace: production
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secretsmanager
    kind: SecretStore
  target:
    name: rds-secret
  data:
    - secretKey: DB_USERNAME
      remoteRef: { key: allsecrets, property: DB_USERNAME }
    - secretKey: DB_PASSWORD
      remoteRef: { key: allsecrets, property: DB_PASSWORD }
    - secretKey: REDIS_AUTH
      remoteRef: { key: allsecrets, property: REDIS_AUTH }
```

Non-sensitive config (like a DB URL) goes in a ConfigMap, not a Secret:

```
bash

kubectl create configmap app-config -n production \
  --from-literal=SPRING_DATASOURCE_URL="jdbc:mysql://your-rds-endpoint.us-east-1.amazonaws.com:3306/your-db-name?serverTimezone=UTC"
```

Phase 5 — Deploy the applications

Manifest layout

```
k8s/
├── backend1-deployment.yaml
├── backend1-service.yaml
├── backend2-deployment.yaml
├── backend2-service.yaml
├── frontend-deployment.yaml
├── frontend-service.yaml
└── frontend-ingress.yaml
```


Kubernetes Service types, at a glance

Type	Reachable from	Provisions	Best for
ClusterIP	Inside cluster only	Internal IP	Internal microservices, backend APIs
NodePort	Outside, via Node IP	Open port on node VMs	Testing/debugging only
LoadBalancer	Public internet	Cloud LB (ALB/NLB)	Production public entry points
ExternalName	Inside cluster	Internal CNAME alias	Pointing pods at external services (RDS, 3rd-party APIs)

Deploy

```
bash

kubectl create namespace production
kubectl apply -f backend1-deployment.yaml
# ...apply the rest, same order as the file list above
```

Apple Silicon Mac gotcha: if you build Docker images on an M-series Mac without specifying the platform, they'll be `arm64` and silently fail to run on EKS's `amd64` nodes. Always build with:

```
bash

docker buildx build \
  --platform linux/amd64 \
  -t <account-id>.dkr.ecr.us-east-1.amazonaws.com/myapp-user-service:latest \
  --push .

kubectl rollout restart deployment user-service -n production
```

Useful debugging commands

```
bash
```

```
# Open a shell inside the cluster network to test connectivity
kubectl run curl-test -n production --image=curlimages/curl --rm -it -- sh

# Test the frontend locally before the ALB is even up
kubectl port-forward -n production svc/frontend 8080:80

# Watch the ALB get provisioned (1-3 minutes)
kubectl get ingress -n production -w
```

Lessons learned (kept as-is — every one of these checks out)

#	Issue	Fix / Lesson
1	Created EKS with incorrect VPC networking understanding.	Learned regions, AZs, VPC, public/private subnet architecture.
2	Used incorrect VPC CIDR initially (/24).	Recreated VPC with /16 for future scaling.
3	NAT Gateway wasn't appearing while creating.	NAT Gateway must be created inside a public subnet with an Elastic IP.
4	Private subnet route tables weren't pointing to NAT Gateway.	Added <code>0.0.0.0/0 → NAT Gateway</code> in private route tables.
5	Confused Internet Gateway and NAT Gateway.	IGW is for public resources; NAT provides outbound-only internet for private resources.
6	Node Group creation failed (<code>Ec2SubnetInvalidConfiguration</code>).	Fixed subnet route tables and networking configuration.
7	Nodes stayed <code>NotReady</code> .	Waited for bootstrap and verified IAM/network configuration.
8	<code>LoadBalancer</code> service remained <code>Pending</code> .	Tagged public subnets correctly for the ALB controller.
9	ALB controller couldn't discover subnets.	Added required subnet tags (<code>kubernetes.io/role/elb</code> , cluster tag).
10	Backend pods entered <code>CrashLoopBackOff</code> .	Investigated logs instead of assuming Kubernetes was broken.

#	Issue	Fix / Lesson
11	Used MySQL running inside Kubernetes.	Planned migration to Amazon RDS for production.
12	Wanted to hardcode database credentials.	Learned to use AWS Secrets Manager instead.
13	Thought pods could read Secrets Manager directly.	Kubernetes can't — needs External Secrets/CSI/SDK.
14	Created the AWS secret only.	Installed External Secrets Operator (ESO).
15	<code>SecretStore</code> CRD not found.	Used correct API version (<code>external-secrets.io/v1</code>).
16	External Secrets couldn't access AWS Secrets Manager.	Configured IRSA.
17	<code>external-secrets</code> ServiceAccount had no IAM role.	Attached IAM role via <code>eksctl create iamserviceaccount</code> .
18	External Secret wasn't syncing.	Created <code>SecretStore</code> and <code>ExternalSecret</code> correctly.
19	Kubernetes Secret created successfully.	Verified sync from Secrets Manager.
20	Deployment YAML had invalid indentation.	Fixed <code>env</code> / <code>valueFrom</code> / <code>secretKeyRef</code> formatting.
21	Password referenced the wrong secret key.	Used <code>DB_PASSWORD</code> instead of <code>DB_USERNAME</code> .
22	Used the old Kubernetes MySQL hostname after moving to RDS.	Replaced with the RDS endpoint.
23	Wanted a dynamic host from Secrets Manager.	Stored <code>DB_HOST</code> in Secrets Manager and synced it.
24	Thought env-var substitution wouldn't work.	<code>\$(DB_HOST)</code> works fine inside <code>env.value</code> .
25	Pods still had no env vars after applying YAML.	The updated manifest had never actually been applied.
26	<code>kubectl describe deployment</code> showed <code>Environment: <none></code> .	Applied the correct manifest.

#	Issue	Fix / Lesson
27	Verified env vars inside the pod.	<code>SPRING_DATASOURCE_URL</code> expanded correctly.
28	App still couldn't connect to the DB.	Realized it was now a networking problem, not a secrets problem.
29	Suspected a wrong password (<code>Communications link failure</code>).	This error usually means TCP/network issues, not bad credentials.
30	RDS security group allowed <code>/24</code> CIDRs.	Worker nodes were actually in <code>/20</code> subnets — traffic was blocked.
31	RDS SG allowed <code>10.0.128.0/24</code> / <code>10.0.144.0/24</code> .	Should've been <code>/20</code> , or better, the EKS Node Security Group directly.
32	Learned the real production lesson.	Prefer Security Group → Security Group rules over CIDR-based rules.